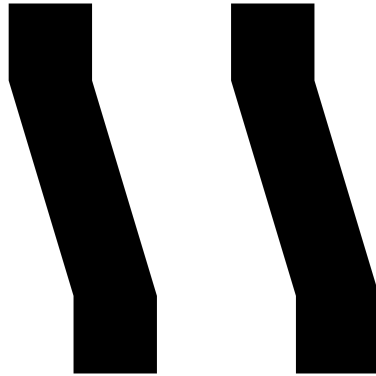




BIP-110: *Activation and Timeline*

Intro

Lukáš Hozda

- Rust/Lisp programmer

Lukáš Hozda

- Rust/Lisp programmer
- Bitcoin, Emacs, HTMX, Linux enthusiast

Lukáš Hozda

- Rust/Lisp programmer
- Bitcoin, Emacs, HTMX, Linux enthusiast
- Teaching Rust at MatFyz, sometimes commercial Rust courses

Lukáš Hozda

- Rust/Lisp programmer
- Bitcoin, Emacs, HTMX, Linux enthusiast
- Teaching Rust at MatFyz, sometimes commercial Rust courses
- Marketing at Braiins

Lukáš Hozda

- Rust/Lisp programmer
- Bitcoin, Emacs, HTMX, Linux enthusiast
- Teaching Rust at MatFyz, sometimes commercial Rust courses
- Marketing at Braiins
- Frequent yapper on twitter @LukasHozda



Building bitcoin in Rust

Written by Lukáš Hozda

With contributors: Daniela Brozzoni, Luukas Pörfors

Today

- How soft fork activations work

Today

- How soft fork activations work
- What BIP-110 changes at the consensus level

Today

- How soft fork activations work
- What BIP-110 changes at the consensus level
- Why its deployment is unusual

Today

- How soft fork activations work
- What BIP-110 changes at the consensus level
- Why its deployment is unusual
- Where we are on the timeline right now

Today

- How soft fork activations work
- What BIP-110 changes at the consensus level
- Why its deployment is unusual
- Where we are on the timeline right now
- How this compares with SegWit and Taproot

Soft forks

Měkké vidličky?

- A tightening of Bitcoin's consensus rules

Měkké vidličky?

- A tightening of Bitcoin's consensus rules
- Some transactions that were valid before become invalid

Měkké vidličky?

- A tightening of Bitcoin's consensus rules
- Some transactions that were valid before become invalid
- Old nodes still accept new blocks as valid

Měkké vidličky?

- A tightening of Bitcoin's consensus rules
- Some transactions that were valid before become invalid
- Old nodes still accept new blocks as valid
- New nodes reject blocks that break the new rules

Měkké vidličky?

- A tightening of Bitcoin's consensus rules
- Some transactions that were valid before become invalid
- Old nodes still accept new blocks as valid
- New nodes reject blocks that break the new rules
- Contrast with a hard fork, which would split the network

Měkké vidličky?

- A tightening of Bitcoin's consensus rules
- Some transactions that were valid before become invalid
- Old nodes still accept new blocks as valid
- New nodes reject blocks that break the new rules
- Contrast with a hard fork, which would split the network
 - HF loosens rules so old nodes reject new nodes

Why activations are tricky

- Miners, node operators, wallets, and exchanges must agree

Why activations are tricky

- Miners, node operators, wallets, and exchanges must agree
- If roughly half the network enforces new rules and half doesn't, you can get two chains

Why activations are tricky

- Miners, node operators, wallets, and exchanges must agree
- If roughly half the network enforces new rules and half doesn't, you can get two chains
- Activation mechanisms exist to avoid that outcome

Why activations are tricky

- Miners, node operators, wallets, and exchanges must agree
- If roughly half the network enforces new rules and half doesn't, you can get two chains
- Activation mechanisms exist to avoid that outcome

Version bits

- Each block header has a 32-bit version field

Version bits

- Each block header has a 32-bit version field
- Bits in that field can be used as flags

Version bits

- Each block header has a 32-bit version field
- Bits in that field can be used as flags
- A bit set means: “this miner is ready for proposal X”

Version bits

- Each block header has a 32-bit version field
- Bits in that field can be used as flags
- A bit set means: “this miner is ready for proposal X”
- Nodes can count how many recent blocks signal a given bit

Version bits

- Each block header has a 32-bit version field
- Bits in that field can be used as flags
- A bit set means: “this miner is ready for proposal X”
- Nodes can count how many recent blocks signal a given bit
- This is how miners coordinate without a central authority

Two families of activation

- MASF: Miner Activated Soft Fork

Two families of activation

- MASF: Miner Activated Soft Fork
- UASF: User Activated Soft Fork

Two families of activation

- MASF: Miner Activated Soft Fork
- UASF: User Activated Soft Fork

MASF

- Miners signal readiness using version bits

MASF

- Miners signal readiness using version bits
- Once a supermajority signals within a retarget window, the fork locks in

MASF

- Miners signal readiness using version bits
- Once a supermajority signals within a retarget window, the fork locks in
 - BIP9 default is 95%, taproot speedy trial was 90%
- Miners effectively hold a veto

MASF

- Miners signal readiness using version bits
- Once a supermajority signals within a retarget window, the fork locks in
 - BIP9 default is 95%, taproot speedy trial was 90%
- Miners effectively hold a veto
- Works well when miners want the change

MASF examples

- BIP65 (CLTV, 2015): adds CheckLockTimeVerify

MASF examples

- BIP65 (CLTV, 2015): adds CheckLockTimeVerify
 - Lets a script require that coins cannot be spent before a given time or block

MASF examples

- BIP65 (CLTV, 2015): adds CheckLockTimeVerify
 - Lets a script require that coins cannot be spent before a given time or block
- BIP68/112/113 (CSV, 2016): adds relative timelocks

MASF examples

- BIP65 (CLTV, 2015): adds CheckLockTimeVerify
 - Lets a script require that coins cannot be spent before a given time or block
- BIP68/112/113 (CSV, 2016): adds relative timelocks
 - Lets a script say “wait N blocks after this coin was mined”

MASF examples

- BIP65 (CLTV, 2015): adds CheckLockTimeVerify
 - Lets a script require that coins cannot be spent before a given time or block
- BIP68/112/113 (CSV, 2016): adds relative timelocks
 - Lets a script say “wait N blocks after this coin was mined”
- Both used the BIP9 framework and activated cleanly

UASF

- Nodes enforce new rules on a flag day regardless of miner signaling

UASF

- Nodes enforce new rules on a flag day regardless of miner signaling
 - Users signal by running a node that enforces the new rules on a flag day

UASF

- Nodes enforce new rules on a flag day regardless of miner signaling
 - Users signal by running a node that enforces the new rules on a flag day
- Non-compliant blocks get rejected by those nodes

UASF

- Nodes enforce new rules on a flag day regardless of miner signaling
 - Users signal by running a node that enforces the new rules on a flag day
- Non-compliant blocks get rejected by those nodes
- Miners either follow or risk building on a chain those nodes ignore

UASF

- Nodes enforce new rules on a flag day regardless of miner signaling
 - Users signal by running a node that enforces the new rules on a flag day
- Non-compliant blocks get rejected by those nodes
- Miners either follow or risk building on a chain those nodes ignore
- Power shifts from miners to economic full nodes

UASF examples

- BIP16 (P2SH, 2012): Pay to Script Hash

UASF examples

- BIP16 (P2SH, 2012): Pay to Script Hash
 - Let users send to the hash of a script, revealing the script later

UASF examples

- BIP16 (P2SH, 2012): Pay to Script Hash
 - Let users send to the hash of a script, revealing the script later
 - Activated by flag day, it did split the chain briefly

UASF examples

- BIP16 (P2SH, 2012): Pay to Script Hash
 - Let users send to the hash of a script, revealing the script later
 - Activated by flag day, it did split the chain briefly
 - Flag day March 31, 2012
- BIP148 (2017): SegWit pressure

UASF examples

- BIP16 (P2SH, 2012): Pay to Script Hash
 - Let users send to the hash of a script, revealing the script later
 - Activated by flag day, it did split the chain briefly
 - Flag day March 31, 2012
- BIP148 (2017): SegWit pressure
 - Would have rejected non-SegWit-signaling blocks after Aug 1, 2017

UASF examples

- BIP16 (P2SH, 2012): Pay to Script Hash
 - Let users send to the hash of a script, revealing the script later
 - Activated by flag day, it did split the chain briefly
 - Flag day March 31, 2012
- BIP148 (2017): SegWit pressure
 - Would have rejected non-SegWit-signaling blocks after Aug 1, 2017
 - It was a threat, not the actual activation mechanism for SegWit

UASF examples

- BIP16 (P2SH, 2012): Pay to Script Hash
 - Let users send to the hash of a script, revealing the script later
 - Activated by flag day, it did split the chain briefly
 - Flag day March 31, 2012
- BIP148 (2017): SegWit pressure
 - Would have rejected non-SegWit-signaling blocks after Aug 1, 2017
 - It was a threat, not the actual activation mechanism for SegWit
- BIP-110

UASF examples

- BIP16 (P2SH, 2012): Pay to Script Hash
 - Let users send to the hash of a script, revealing the script later
 - Activated by flag day, it did split the chain briefly
 - Flag day March 31, 2012
- BIP148 (2017): SegWit pressure
 - Would have rejected non-SegWit-signaling blocks after Aug 1, 2017
 - It was a threat, not the actual activation mechanism for SegWit
- BIP-110
 - Miners cooperate by setting bit 4 or their blocks will be invalid

BIP9

- The standard MASF framework used from 2015 onward

BIP9

- The standard MASF framework used from 2015 onward
- Signaling window is broken into 2016-block retarget periods

BIP9

- The standard MASF framework used from 2015 onward
- Signaling window is broken into 2016-block retarget periods
- 95% of blocks in a period must signal for lock-in

BIP9

- The standard MASF framework used from 2015 onward
- Signaling window is broken into 2016-block retarget periods
- 95% of blocks in a period must signal for lock-in
- Has a start time and a timeout (can end in FAILED)

BIP9

- The standard MASF framework used from 2015 onward
- Signaling window is broken into 2016-block retarget periods
- 95% of blocks in a period must signal for lock-in
- Has a start time and a timeout (can end in FAILED)
- Miners can veto by simply not signaling

BIP9 states

- DEFINED: waiting for start time

BIP9 states

- DEFINED: waiting for start time
- STARTED: signaling window is open

BIP9 states

- **DEFINED**: waiting for start time
- **STARTED**: signaling window is open
- **LOCKED_IN**: threshold was reached, activation scheduled

BIP9 states

- DEFINED: waiting for start time
- STARTED: signaling window is open
- LOCKED_IN: threshold was reached, activation scheduled
- ACTIVE: new rules are enforced

BIP9 states

- DEFINED: waiting for start time
- STARTED: signaling window is open
- LOCKED_IN: threshold was reached, activation scheduled
- ACTIVE: new rules are enforced
- FAILED: timeout hit without reaching threshold

BIP8

- A BIP9 variant, proposed after the SegWit stalemate

BIP8

- A BIP9 variant, proposed after the SegWit stalemate
- Uses block heights instead of wall-clock time

BIP8

- A BIP9 variant, proposed after the SegWit stalemate
- Uses block heights instead of wall-clock time
- Adds a LOT (Lock on Timeout) parameter:

BIP8

- A BIP9 variant, proposed after the SegWit stalemate
- Uses block heights instead of wall-clock time
- Adds a LOT (Lock on Timeout) parameter:
 - LOT=false behaves like BIP9

BIP8

- A BIP9 variant, proposed after the SegWit stalemate
- Uses block heights instead of wall-clock time
- Adds a LOT (Lock on Timeout) parameter:
 - LOT=false behaves like BIP9
 - LOT=true guarantees activation at timeout even without miner support

BIP8

- A BIP9 variant, proposed after the SegWit stalemate
- Uses block heights instead of wall-clock time
- Adds a LOT (Lock on Timeout) parameter:
 - LOT=false behaves like BIP9
 - LOT=true guarantees activation at timeout even without miner support
- Never activated a real deployment, but influenced later designs

Speedy Trial

- A compromise design used for Taproot in 2021

Speedy Trial

- A compromise design used for Taproot in 2021
- BIP8 with LOT=false and a short window (3 months)

Speedy Trial

- A compromise design used for Taproot in 2021
- BIP8 with LOT=false and a short window (3 months)
- 90% threshold instead of 95%

Speedy Trial

- A compromise design used for Taproot in 2021
- BIP8 with LOT=false and a short window (3 months)
- 90% threshold instead of 95%
- If miners don't cooperate, it fails fast with low drama

Speedy Trial

- A compromise design used for Taproot in 2021
- BIP8 with LOT=false and a short window (3 months)
- 90% threshold instead of 95%
- If miners don't cooperate, it fails fast with low drama
- If they do, activation is quick

Taproot

- BIPs 340, 341, 342 (Schnorr signatures + Taproot + Tapscript)

Taproot

- BIPs 340, 341, 342 (Schnorr signatures + Taproot + Tapscript)
- Activated via Speedy Trial in November 2021

Taproot

- BIPs 340, 341, 342 (Schnorr signatures + Taproot + Tapscript)
- Activated via Speedy Trial in November 2021
- Signaling reached near 100% in the lock-in period

Taproot

- BIPs 340, 341, 342 (Schnorr signatures + Taproot + Tapscript)
- Activated via Speedy Trial in November 2021
- Signaling reached near 100% in the lock-in period
- The current baseline for “how Bitcoin upgrades are done”

Mechanisms summary

Mechanism	Miner veto	Example
BIP9	Yes	CLTV, CSV, SegWit
BIP8 LOT=false	Yes	Taproot (Speedy Trial)
BIP8 LOT=true	No	Kinda Bip-110
Flag-day UASF	No	BIP16 (P2SH), BIP148 (threat)

BIP-110

High-level goal

- Push back against non-monetary data embedding at the consensus level

High-level goal

- Push back against non-monetary data embedding at the consensus level
- Temporarily invalidate several ways of embedding large payloads

High-level goal

- Push back against non-monetary data embedding at the consensus level
- Temporarily invalidate several ways of embedding large payloads
- One-year sunset: rules automatically expire

High-level goal

- Push back against non-monetary data embedding at the consensus level
- Temporarily invalidate several ways of embedding large payloads
- One-year sunset: rules automatically expire
- Stated intent is to “send a message” more than to fix data embedding forever

Terms

- Before listing the restrictions, we need definitions

Terms

- Before listing the restrictions, we need definitions
- scriptPubKey, PUSHDATA, witness data, taproot annex, control block, tapscript opcodes

Terms

- Before listing the restrictions, we need definitions
- scriptPubKey, PUSHDATA, witness data, taproot annex, control block, tapscript opcodes
- These come straight from Bitcoin's script system

Terms

- Before listing the restrictions, we need definitions
- scriptPubKey, PUSHDATA, witness data, taproot annex, control block, tapscript opcodes
- These come straight from Bitcoin's script system

If you don't know, scripts are essentially mini programs attached to coins

scriptPubKey

- The “locking script” on each transaction output

scriptPubKey

- The “locking script” on each transaction output
- Defines the conditions to spend those coins

scriptPubKey

- The “locking script” on each transaction output
- Defines the conditions to spend those coins
- Modern addresses produce small scriptPubKeys (often 22 to 34 bytes)

scriptPubKey

- The “locking script” on each transaction output
- Defines the conditions to spend those coins
- Modern addresses produce small scriptPubKeys (often 22 to 34 bytes)
 - Quantum resistant pubkeys are much bigger, but you’d commit via hash

scriptPubKey

- The “locking script” on each transaction output
- Defines the conditions to spend those coins
- Modern addresses produce small scriptPubKeys (often 22 to 34 bytes)
 - Quantum resistant pubkeys are much bigger, but you’d commit via hash
 - Commit now reveal later is a better idea anyway and we use it (P2PKH and onwards)

scriptPubKey

- The “locking script” on each transaction output
- Defines the conditions to spend those coins
- Modern addresses produce small scriptPubKeys (often 22 to 34 bytes)
 - Quantum resistant pubkeys are much bigger, but you’d commit via hash
 - Commit now reveal later is a better idea anyway and we use it (P2PKH and onwards)
- Every unspent output’s scriptPubKey lives in the UTXO set forever

scriptPubKey

- The “locking script” on each transaction output
- Defines the conditions to spend those coins
- Modern addresses produce small scriptPubKeys (often 22 to 34 bytes)
 - Quantum resistant pubkeys are much bigger, but you’d commit via hash
 - Commit now reveal later is a better idea anyway and we use it (P2PKH and onwards)
- Every unspent output’s scriptPubKey lives in the UTXO set forever
 - Embedding data here does bloat the UTXO set

scriptPubKey

- The “locking script” on each transaction output
- Defines the conditions to spend those coins
- Modern addresses produce small scriptPubKeys (often 22 to 34 bytes)
 - Quantum resistant pubkeys are much bigger, but you’d commit via hash
 - Commit now reveal later is a better idea anyway and we use it (P2PKH and onwards)
- Every unspent output’s scriptPubKey lives in the UTXO set forever
 - Embedding data here does bloat the UTXO set

PUSHDATA

- Script opcodes that push bytes onto the execution stack

PUSHDATA

- Script opcodes that push bytes onto the execution stack
- OP_PUSHDATA1, 2, and 4 take 1, 2, or 4-byte length prefixes

PUSHDATA

- Script opcodes that push bytes onto the execution stack
- OP_PUSHDATA1, 2, and 4 take 1, 2, or 4-byte length prefixes
- Used for legitimate things: public keys, signatures, hashes

PUSHDATA

- Script opcodes that push bytes onto the execution stack
- OP_PUSHDATA1, 2, and 4 take 1, 2, or 4-byte length prefixes
- Used for legitimate things: public keys, signatures, hashes
- Also the main vehicle for inscribing arbitrary data inside scripts

Inscription envelope

<32-byte x-only pubkey>

OP_CHECKSIG

OP_FALSE

OP_IF

OP_PUSHBYTES_3 "ord"

OP_PUSHDAT_1

OP_PUSHBYTES_9 "image/png"

OP_0

OP_PUSHDAT_2 <520 bytes>

OP_PUSHDAT_2 <520 bytes>

OP_PUSHDAT_2 <520 bytes>

OP_PUSHDATA2 <440 bytes>

OP_ENDIF

OP_PUSHBYTES is not a real opcode, it's a raw direct push of 1-75 bytes

Witness data

- Added by SegWit in 2017

Witness data

- Added by SegWit in 2017
- A separate area of the transaction for signatures and script arguments

Witness data

- Added by SegWit in 2017
- A separate area of the transaction for signatures and script arguments
- Weighed at 1 unit per byte instead of 4 (the “witness discount”)

Witness data

- Added by SegWit in 2017
- A separate area of the transaction for signatures and script arguments
- Weighed at 1 unit per byte instead of 4 (the “witness discount”)
- This discount is why inscriptions live there: it’s the cheapest space

Witness data

- Added by SegWit in 2017
- A separate area of the transaction for signatures and script arguments
- Weighed at 1 unit per byte instead of 4 (the “witness discount”)
- This discount is why inscriptions live there: it’s the cheapest space
- Post-Taproot, witness data can include scripts, signatures, or arbitrary payloads

Tapscript

- The scripting language used inside a Taproot script-path spend

Tapscript

- The scripting language used inside a Taproot script-path spend
 - FYI also has a key-path spend
- Each script-path alternative is called a Tapleaf

Tapscript

- The scripting language used inside a Taproot script-path spend
 - FYI also has a key-path spend
- Each script-path alternative is called a Tapleaf
- All of a user's Tapleaves together form a Merkle tree called the Taptree

Tapscript

- The scripting language used inside a Taproot script-path spend
 - FYI also has a key-path spend
- Each script-path alternative is called a Tapleaf
- All of a user's Tapleaves together form a Merkle tree called the Taptree
- Only the leaf you actually spend needs to be revealed

Tapscript

- The scripting language used inside a Taproot script-path spend
 - FYI also has a key-path spend
- Each script-path alternative is called a Tapleaf
- All of a user's Tapleaves together form a Merkle tree called the Taptree
- Only the leaf you actually spend needs to be revealed
 - No scripts will be revealed if you use the script-path spend

Control blocks

- Part of a Taproot script-path spend

Control blocks

- Part of a Taproot script-path spend
- Proves that the revealed Tapleaf is really part of the committed Taptree

Control blocks

- Part of a Taproot script-path spend
- Proves that the revealed Tapleaf is really part of the committed Taptree
- Contains the internal public key, parity bit, and a Merkle path

Control blocks

- Part of a Taproot script-path spend
- Proves that the revealed Tapleaf is really part of the committed Taptree
- Contains the internal public key, parity bit, and a Merkle path
- Size grows with tree depth: 33 bytes + 32 bytes per level

Control blocks

- Part of a Taproot script-path spend
- Proves that the revealed Tapleaf is really part of the committed Taptree
- Contains the internal public key, parity bit, and a Merkle path
- Size grows with tree depth: 33 bytes + 32 bytes per level
- Size of the control block limits how deep the Taptree can be

Control blocks

- Part of a Taproot script-path spend
- Proves that the revealed Tapleaf is really part of the committed Taptree
- Contains the internal public key, parity bit, and a Merkle path
- Size grows with tree depth: 33 bytes + 32 bytes per level
- Size of the control block limits how deep the Taptree can be
- Can be used to embed arbitrary data by making the taptree lopsided and using arbitrary 32 byte siblings

Taproot annex

- An optional extra element in a Taproot witness stack

Taproot annex

- An optional extra element in a Taproot witness stack
- Reserved for future upgrades, currently has no defined meaning

Taproot annex

- An optional extra element in a Taproot witness stack
- Reserved for future upgrades, currently has no defined meaning
- No size limit in the current rules

Taproot annex

- An optional extra element in a Taproot witness stack
- Reserved for future upgrades, currently has no defined meaning
- No size limit in the current rules
- Today, wallets don't use it AFAIK

Tapscript opcodes we care about

- OP_SUCCESS*: reserved opcodes for future soft forks

Tapscript opcodes we care about

- OP_SUCCESS*: reserved opcodes for future soft forks
 - If encountered, the script succeeds unconditionally

Tapscript opcodes we care about

- OP_SUCCESS*: reserved opcodes for future soft forks
 - If encountered, the script succeeds unconditionally
- OP_IF / OP_NOTIF: conditional branching inside a script

Tapscript opcodes we care about

- OP_SUCCESS*: reserved opcodes for future soft forks
 - If encountered, the script succeeds unconditionally
- OP_IF / OP_NOTIF: conditional branching inside a script
 - Execute one branch if a value is true, another if false

Tapscript opcodes we care about

- OP_SUCCESS*: reserved opcodes for future soft forks
 - If encountered, the script succeeds unconditionally
- OP_IF / OP_NOTIF: conditional branching inside a script
 - Execute one branch if a value is true, another if false
- All three can be abused to stuff data into scripts that never executes

BIP-110 restrictions

- $\text{scriptPubKey} \leq 34$ bytes (OP_RETURN up to 83)

BIP-110 restrictions

- $\text{scriptPubKey} \leq 34$ bytes (OP_RETURN up to 83)
- PUSHDATA payloads and script argument witness items ≤ 256 bytes

BIP-110 restrictions

- `scriptPubKey` $\leq$ 34 bytes (OP_RETURN up to 83)
- PUSHDATA payloads and script argument witness items $\leq$ 256 bytes
- No spending of undefined witness or Tapleaf versions

BIP-110 restrictions

- `scriptPubKey` $\leq$ 34 bytes (OP_RETURN up to 83)
- PUSHDATA payloads and script argument witness items $\leq$ 256 bytes
- No spending of undefined witness or Tapleaf versions
- No Taproot annex in witness stacks

BIP-110 restrictions

- $\text{scriptPubKey} \leq 34$ bytes (OP_RETURN up to 83)
- PUSHDATA payloads and script argument witness items ≤ 256 bytes
- No spending of undefined witness or Tapleaf versions
- No Taproot annex in witness stacks
- Control blocks ≤ 257 bytes (max 128 leaves)

BIP-110 restrictions

- $\text{scriptPubKey} \leq 34$ bytes (OP_RETURN up to 83)
- PUSHDATA payloads and script argument witness items ≤ 256 bytes
- No spending of undefined witness or Tapleaf versions
- No Taproot annex in witness stacks
- Control blocks ≤ 257 bytes (max 128 leaves)
- No OP_SUCCESS*, OP_IF, or OP_NOTIF in tapscripts

- Caps the size of individual data chunks to 256 bytes

- Caps the size of individual data chunks to 256 bytes
- Closes off large Taptrees beyond 128 leaves

- Caps the size of individual data chunks to 256 bytes
- Closes off large Taptrees beyond 128 leaves
- Bans two conditional opcodes often used to skip over embedded data

- Caps the size of individual data chunks to 256 bytes
- Closes off large Taptrees beyond 128 leaves
- Bans two conditional opcodes often used to skip over embedded data
- Locks down parts of the protocol reserved for future upgrades

- Caps the size of individual data chunks to 256 bytes
- Closes off large Taptrees beyond 128 leaves
- Bans two conditional opcodes often used to skip over embedded data
- Locks down parts of the protocol reserved for future upgrades
- Leaves normal payments untouched

Deployment parameters

- name: `reduced_data`

Deployment parameters

- name: reduced_data
- bit: 4

Deployment parameters

- name: reduced_data
- bit: 4
- starttime: December 2025

Deployment parameters

- name: reduced_data
- bit: 4
- starttime: December 2025
- timeout: NO_TIMEOUT

Deployment parameters

- name: reduced_data
- bit: 4
- starttime: December 2025
- timeout: NO_TIMEOUT
- max_activation_height: 965664 (September 2026)

Deployment parameters

- name: reduced_data
- bit: 4
- starttime: December 2025
- timeout: NO_TIMEOUT
- max_activation_height: 965664 (September 2026)
- active_duration: 52416 blocks (1 year)

Deployment parameters

- name: reduced_data
- bit: 4
- starttime: December 2025
- timeout: NO_TIMEOUT
- max_activation_height: 965664 (September 2026)
- active_duration: 52416 blocks (1 year)
- threshold: $1109/2016 = 55\%$

Deviations from BIP9

1. Threshold is 55% instead of 95%
- 2.
- 3.
- 4.
- 5.

Deviations from BIP9

1. Threshold is 55% instead of 95%
2. No timeout: uses max_activation_height instead
- 3.
- 4.
- 5.

Deviations from BIP9

1. Threshold is 55% instead of 95%
2. No timeout: uses max_activation_height instead
3. Mandatory signaling period before max_activation_height
- 4.
- 5.

Deviations from BIP9

1. Threshold is 55% instead of 95%
2. No timeout: uses `max_activation_height` instead
3. Mandatory signaling period before `max_activation_height`
4. Auto-expires after `active_duration` (1 year of ACTIVE)
- 5.

Deviations from BIP9

1. Threshold is 55% instead of 95%
2. No timeout: uses `max_activation_height` instead
3. Mandatory signaling period before `max_activation_height`
4. Auto-expires after `active_duration` (1 year of ACTIVE)
5. Introduces a new terminal state, EXPIRED

State machine

- DEFINED $\rightarrow$ STARTED (at starttime)

State machine

- DEFINED $\rightarrow$ STARTED (at starttime)
- STARTED $\rightarrow$ LOCKED_IN (threshold hit, or forced by mandatory signaling)

State machine

- DEFINED → STARTED (at starttime)
- STARTED → LOCKED_IN (threshold hit, or forced by mandatory signaling)
- LOCKED_IN → ACTIVE (one retarget period later)

State machine

- DEFINED → STARTED (at starttime)
- STARTED → LOCKED_IN (threshold hit, or forced by mandatory signaling)
- LOCKED_IN → ACTIVE (one retarget period later)
- ACTIVE → EXPIRED (after about one year)

State machine

- DEFINED → STARTED (at starttime)
- STARTED → LOCKED_IN (threshold hit, or forced by mandatory signaling)
- LOCKED_IN → ACTIVE (one retarget period later)
- ACTIVE → EXPIRED (after about one year)
- FAILED is never reached: deployment cannot fail by timeout

Why this combination is unusual

- 55% is well below recent precedent

Why this combination is unusual

- 55% is well below recent precedent
- Mandatory signaling means activation is effectively built in

Why this combination is unusual

- 55% is well below recent precedent
- Mandatory signaling means activation is effectively built in
- It is the first soft fork designed to expire automatically

Why this combination is unusual

- 55% is well below recent precedent
- Mandatory signaling means activation is effectively built in
- It is the first soft fork designed to expire automatically
- The window is compressed (9 months)

Why this combination is unusual

- 55% is well below recent precedent
- Mandatory signaling means activation is effectively built in
- It is the first soft fork designed to expire automatically
- The window is compressed (9 months)

Threshold context

Fork	Threshold	Mechanism
CLTV, CSV	95%	BIP9
SegWit	95%	BIP9 (later 80% via BIP91)
BIP91	80%	BIP9 variant
Taproot	90%	Speedy Trial
BIP-110	55%	Modified BIP9 with BIP8-style forcing

Timeline

Key heights

- starttime: Dec 1, 2025

Key heights

- starttime: Dec 1, 2025
- Mandatory signaling window: blocks 961,632 to 963,647

Key heights

- starttime: Dec 1, 2025
- Mandatory signaling window: blocks 961,632 to 963,647
- Latest possible lock-in: block 963,648

Key heights

- starttime: Dec 1, 2025
- Mandatory signaling window: blocks 961,632 to 963,647
- Latest possible lock-in: block 963,648
- Activation: block 965,664 (Sep 2026)

Key heights

- starttime: Dec 1, 2025
- Mandatory signaling window: blocks 961,632 to 963,647
- Latest possible lock-in: block 963,648
- Activation: block 965,664 (Sep 2026)
- Expiry: block 1,018,080 (Sep 2027)

Where we are now

- Current block height: 946,182 (as of April 22, 2026)

Where we are now

- Current block height: 946,182 (as of April 22, 2026)
- Current retarget period: 469

Where we are now

- Current block height: 946,182 (as of April 22, 2026)
- Current retarget period: 469
- Roughly 8 retarget periods until mandatory signaling begins

Where we are now

- Current block height: 946,182 (as of April 22, 2026)
- Current retarget period: 469
- Roughly 8 retarget periods until mandatory signaling begins
- That is about 16 weeks, give or take difficulty adjustments

Observed signaling so far

Period	Range	Signaling
465	937440–939455	0.05% (1/2016)
466	939456–941471	0%
467	941472–943487	0.15% (3/2016)
468	943488–945503	0.10% (2/2016)
469	945504–947519	0% so far

Signalling

- Required: 1109 signaling blocks in a retarget period

Signalling

- Required: 1109 signaling blocks in a retarget period
- Observed so far: 0 to 3 signaling blocks per period

Signalling

- Required: 1109 signaling blocks in a retarget period
- Observed so far: 0 to 3 signaling blocks per period
- Natural lock-in through organic signaling looks very unlikely

Signalling

- Required: 1109 signaling blocks in a retarget period
- Observed so far: 0 to 3 signaling blocks per period
- Natural lock-in through organic signaling looks very unlikely
- The practical activation path is mandatory signaling

Signalling

- Required: 1109 signaling blocks in a retarget period
- Observed so far: 0 to 3 signaling blocks per period
- Natural lock-in through organic signaling looks very unlikely
- The practical activation path is mandatory signaling
- That only matters if enough nodes and pools actually run the rule

Two paths to activation

1. Natural: some retarget period reaches 55% → LOCKED_IN
- 2.

Two paths to activation

1. Natural: some retarget period reaches 55% → LOCKED_IN
2. Forced: during the mandatory signaling window, non-signaling blocks are rejected

Two paths to activation

1. Natural: some retarget period reaches 55% → LOCKED_IN
2. Forced: during the mandatory signaling window, non-signaling blocks are rejected

Two paths to activation

1. Natural: some retarget period reaches 55% → LOCKED_IN
 2. Forced: during the mandatory signaling window, non-signaling blocks are rejected
- Both paths converge on ACTIVE at block 965,664

Two paths to activation

1. Natural: some retarget period reaches 55% → LOCKED_IN
 2. Forced: during the mandatory signaling window, non-signaling blocks are rejected
- Both paths converge on ACTIVE at block 965,664
 - The only way to stop activation is for the client to not be adopted

SegWit

SegWit comparison

- Most recent contested activation

SegWit comparison

- Most recent contested activation
- Illustrated all three dynamics: MASF stall, UASF pressure, miner compromise

SegWit comparison

- Most recent contested activation
- Illustrated all three dynamics: MASF stall, UASF pressure, miner compromise
- Led directly to the design of BIP8 and Speedy Trial

SegWit comparison

- Most recent contested activation
- Illustrated all three dynamics: MASF stall, UASF pressure, miner compromise
- Led directly to the design of BIP8 and Speedy Trial
- Good mirror for thinking about BIP-110

SegWit (BIP141) initial deployment

- Activation: BIP9, 95% threshold

SegWit (BIP141) initial deployment

- Activation: BIP9, 95% threshold
- Signaling window: Nov 15, 2016 to Nov 15, 2017

SegWit (BIP141) initial deployment

- Activation: BIP9, 95% threshold
- Signaling window: Nov 15, 2016 to Nov 15, 2017
- Used bit 1 in the block version field

SegWit (BIP141) initial deployment

- Activation: BIP9, 95% threshold
- Signaling window: Nov 15, 2016 to Nov 15, 2017
- Used bit 1 in the block version field
- 26 retarget periods available

The stall

- Miner signaling stuck around 30% for months

The stall

- Miner signaling stuck around 30% for months
- A handful of large pools actively opposed it

The stall

- Miner signaling stuck around 30% for months
- A handful of large pools actively opposed it
- BIP9's 5% veto margin was easy to hit

The stall

- Miner signaling stuck around 30% for months
- A handful of large pools actively opposed it
- BIP9's 5% veto margin was easy to hit
- Users and businesses grew increasingly frustrated

BIP148 (the UASF threat)

- Proposed flag day: August 1, 2017

BIP148 (the UASF threat)

- Proposed flag day: August 1, 2017
- BIP148 nodes would have rejected blocks that didn't signal SegWit

BIP148 (the UASF threat)

- Proposed flag day: August 1, 2017
- BIP148 nodes would have rejected blocks that didn't signal SegWit
- A user-driven threat, not a developer consensus

BIP148 (the UASF threat)

- Proposed flag day: August 1, 2017
- BIP148 nodes would have rejected blocks that didn't signal SegWit
- A user-driven threat, not a developer consensus
- Created a real possibility of chain split

BIP91 (the compromise)

- 80% threshold over a short 336-block window

BIP91 (the compromise)

- 80% threshold over a short 336-block window
- Once active, orphaned any block not signaling SegWit

BIP91 (the compromise)

- 80% threshold over a short 336-block window
- Once active, orphaned any block not signaling SegWit
- Came out of the New York Agreement between miners and businesses

BIP91 (the compromise)

- 80% threshold over a short 336-block window
- Once active, orphaned any block not signaling SegWit
- Came out of the New York Agreement between miners and businesses
- Locked in and activated before BIP148's flag day

BIP91 (the compromise)

- 80% threshold over a short 336-block window
- Once active, orphaned any block not signaling SegWit
- Came out of the New York Agreement between miners and businesses
- Locked in and activated before BIP148's flag day
- This is what actually triggered SegWit's lock-in

Important nuance

- People often say “SegWit was activated by UASF”

Important nuance

- People often say “SegWit was activated by UASF”
- That’s not quite right

Important nuance

- People often say “SegWit was activated by UASF”
- That’s not quite right
- BIP148 (UASF) was a threat that changed incentives

Important nuance

- People often say “SegWit was activated by UASF”
- That’s not quite right
- BIP148 (UASF) was a threat that changed incentives
- BIP91 (MASF variant) did the actual forcing work on miners

Important nuance

- People often say “SegWit was activated by UASF”
- That’s not quite right
- BIP148 (UASF) was a threat that changed incentives
- BIP91 (MASF variant) did the actual forcing work on miners
- SegWit’s own BIP9 deployment then reached 100% signaling under pressure

SegWit timeline

- Nov 15, 2016: BIP9 signaling begins

SegWit timeline

- Nov 15, 2016: BIP9 signaling begins
- Mar to Jun 2017: stalemate near 30%

SegWit timeline

- Nov 15, 2016: BIP9 signaling begins
- Mar to Jun 2017: stalemate near 30%
- Mar 12, 2017: BIP148 proposed

SegWit timeline

- Nov 15, 2016: BIP9 signaling begins
- Mar to Jun 2017: stalemate near 30%
- Mar 12, 2017: BIP148 proposed
- May 22, 2017: BIP91 proposed

SegWit timeline

- Nov 15, 2016: BIP9 signaling begins
- Mar to Jun 2017: stalemate near 30%
- Mar 12, 2017: BIP148 proposed
- May 22, 2017: BIP91 proposed
- Jul 2017: BIP91 locked in and activated

SegWit timeline

- Nov 15, 2016: BIP9 signaling begins
- Mar to Jun 2017: stalemate near 30%
- Mar 12, 2017: BIP148 proposed
- May 22, 2017: BIP91 proposed
- Jul 2017: BIP91 locked in and activated
- Aug 9, 2017: SegWit locked in

SegWit timeline

- Nov 15, 2016: BIP9 signaling begins
- Mar to Jun 2017: stalemate near 30%
- Mar 12, 2017: BIP148 proposed
- May 22, 2017: BIP91 proposed
- Jul 2017: BIP91 locked in and activated
- Aug 9, 2017: SegWit locked in
- Aug 24, 2017: SegWit activated at block 481,824

Side by side

	SegWit	Taproot	BIP-110
Threshold	95%	90%	55%
Window	1 year	3 months	9 months
Forced activation	No (only threat)	No	Yes (built-in)
Permanent	Yes	Yes	No (1 year)
Peak signaling	Reached 100%	99%	<0.2% so far

Where we are

Where we are

Current state

- BIP-110 status: Draft

Current state

- BIP-110 status: Draft
- Implemented in: Bitcoin Knots

Current state

- BIP-110 status: Draft
- Implemented in: Bitcoin Knots
- Not implemented in: Bitcoin Core

Current state

- BIP-110 status: Draft
- Implemented in: Bitcoin Knots
- Not implemented in: Bitcoin Core
- Observed pool signaling: effectively zero

Current state

- BIP-110 status: Draft
- Implemented in: Bitcoin Knots
- Not implemented in: Bitcoin Core
- Observed pool signaling: effectively zero
- No public commitments from major mining pools

Possible scenarios

- A: Pools reach 55% in some retarget period → natural lock-in

Possible scenarios

- A: Pools reach 55% in some retarget period → natural lock-in
- B: Mandatory signaling forces activation mid-2026

Possible scenarios

- A: Pools reach 55% in some retarget period → natural lock-in
- B: Mandatory signaling forces activation mid-2026
- C: The client is not adopted, the deployment has no network-wide effect

Possible scenarios

- A: Pools reach 55% in some retarget period → natural lock-in
- B: Mandatory signaling forces activation mid-2026
- C: The client is not adopted, the deployment has no network-wide effect
- From the rest of the ecosystem's perspective, C is indistinguishable from the BIP not existing

Where we are

Open questions

- Will Bitcoin Core merge an implementation?

Open questions

- Will Bitcoin Core merge an implementation?
 - PR closed: <https://github.com/bitcoin/bitcoin/pull/34930>

Open questions

- Will Bitcoin Core merge an implementation?
 - PR closed: <https://github.com/bitcoin/bitcoin/pull/34930>
- Will any major mining pool run it?

Open questions

- Will Bitcoin Core merge an implementation?
 - PR closed: <https://github.com/bitcoin/bitcoin/pull/34930>
- Will any major mining pool run it?
- What does “activation” mean if almost no one enforces it?

Open questions

- Will Bitcoin Core merge an implementation?
 - PR closed: <https://github.com/bitcoin/bitcoin/pull/34930>
- Will any major mining pool run it?
- What does “activation” mean if almost no one enforces it?
- Does mandatory signaling matter if the rule-checking nodes are a tiny minority?

Differences from SegWit

- SegWit: wide user demand, miner resistance

Differences from SegWit

- SegWit: wide user demand, miner resistance
- BIP-110: vocal minority support, no visible miner or Core support

Differences from SegWit

- SegWit: wide user demand, miner resistance
- BIP-110: vocal minority support, no visible miner or Core support
- SegWit: activation came from public negotiation and pressure

Differences from SegWit

- SegWit: wide user demand, miner resistance
- BIP-110: vocal minority support, no visible miner or Core support
- SegWit: activation came from public negotiation and pressure
- BIP-110: forced activation is written directly into the deployment

Differences from SegWit

- SegWit: wide user demand, miner resistance
- BIP-110: vocal minority support, no visible miner or Core support
- SegWit: activation came from public negotiation and pressure
- BIP-110: forced activation is written directly into the deployment

End

End

Q&A